

Q9

	0	1	2	3	4	5	6	7
arr[] =	73	74	75	71	69	72	76	73
	↑	↑	↑	↑	↑	↑	↑	↑
days →	1	1	4	2	1	1	0	0
	↑	↑						

~~ans = [1, 1, 4, 2, 1, 1, 0, 0]~~

answer [1 | 1 | 4 | 2 | 1 | 1 | 0 | 0]

73	6 - 2 = 4
76	6 - 5 = 1
72	5 - 4 = 1
69	5 - 3 = 2
71	6 - 2 = 4
74	2 - 1 = 1
75	1 - 0 = 1
74	

stack, index.

Route Force

→ hme change next warmer day.

ie.

say 71 se warmer day 74 hai.
toh, [74 ka index - 71 ka index]

ie. 1 - 0

= 1 → day → which is the answer.

Uy. For 74.

74 se warmer day 75 hai.

toh → [75 ka index - 74 ka index.]

[2 - 1]

= 1 → day → answer

25 is warmer day. 71 69 72 76.
4th → [76 to index - 25 to index]

[6 - 2]

→ 4. → days. Ans

// code:

```
int answer[] = new int[n];  
for (int i = 0; i < temp.length; i++) {  
    for (int j = i+1; j < temp.length; j++) {  
        if (temp[j] > temp[i]) {  
            answer[i] = j - i;  
            break;  
        }  
    }  
}  
return answer;
```

Time Complexity $O(n^2)$.

⇒ For Optimize this we use Stack
data structures :-

// code:-

```
int n = temperatures.length;
int[] ans = new int[n];
Stack<Integer> st = new Stack<>();

for (int i = n-1; i >= 0; i--) {
    while (!st.isEmpty() & temperatures
           [st.peek()] <= temperatures[i]) {
        st.pop();
    }
    if (!st.isEmpty()) {
        ans[i] = st.peek() - i;
    }
    st.push(i);
}
return ans;
}
```

Using stack:-

→ optimum approach:-

→ time complexity → $O(n)$